

dbVybe: No-code, Multi-Agent Database Exploration Tool

Problem Statement

Database exploration is a critical aspect of managing and maintaining any system's data. Simply having a database is not enough, organizations must be able to understand, navigate, and interpret the information it contains. However, most database systems require significant technical expertise to explore and analyze data effectively. This presents a challenge for business owners, stakeholders, and clients without a technical background, making data-driven decision-making more difficult and less accessible.

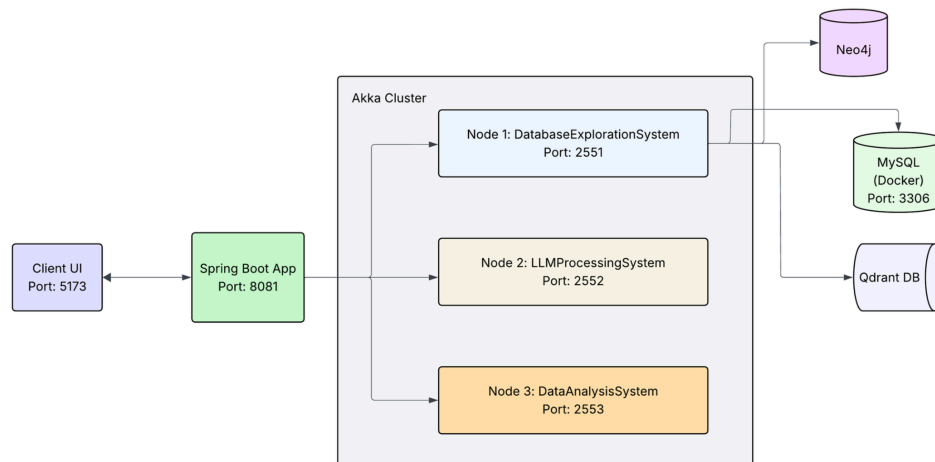
Solution to the Challenge

Presenting **dbVybe**, a no-code, multi-agent database exploration tool which allows users to connect a database of their choice and explore it using natural language

By eliminating the need for technical expertise, it empowers users from all backgrounds to seamlessly explore, query, and analyze their data, making insights accessible without prior database or query language knowledge.

Architecture Overview

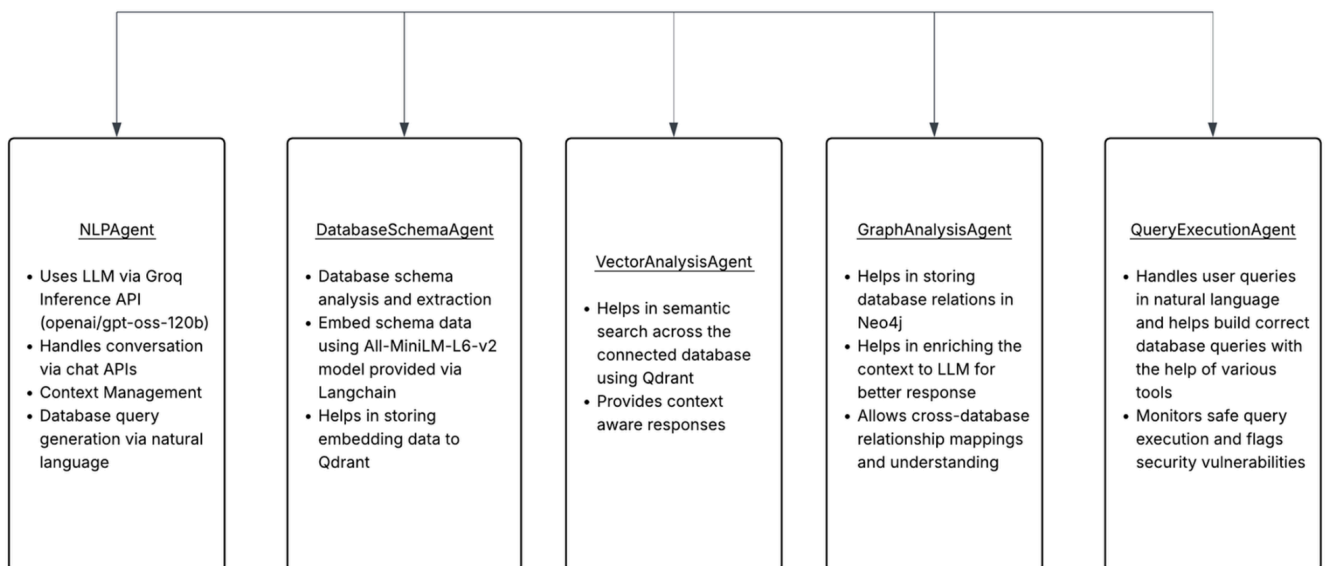
- Distributed architecture using **Akka Cluster** with **3 specialized nodes**
- **Java Spring Boot** web framework for **REST API** implementation
- **React** and **Typescript** for frontend
- **Akka Actor Model** for concurrency
- **Multi-Agent System** for intelligent tool usage and database operations
- LLM access via **Groq Inference Engine APIs** (using *openai/gpt-oss-120b*)



The application flow starts with the user requests coming in from the Client UI via REST APIs to the Spring Boot App acting as the web-client on the backend. The Spring Boot app is connected to the Akka Cluster via ClusterManager which is responsible for handling the requests and routing them to the appropriate nodes via Akka Messaging. Requests are then handled as messages and passed around by the ActorSystems (nodes) and the various Actors defined in the cluster. Actors communicate with various tools such as MySQL database, Qdrant vector database, Neo4j graph database, all-miniLM embedding model via Langchain, LLM model via Groq inference engine and other intermediate services to perform a range of operations on the connected database.

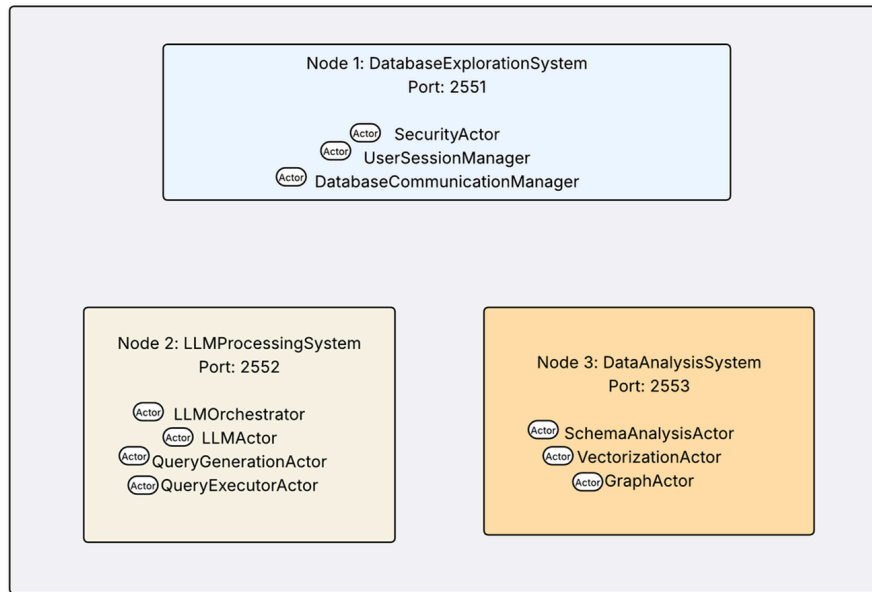
Catalog of AI Agents

Below image shows all the AI agents present in the **dbVybe** architecture and highlights their key functionalities and responsibilities.



Actor Hierarchy in Akka Cluster

The dbVybe architecture relies heavily on the Actor Model for concurrency and distributed architecture. dbVybe is designed with Akka Cluster architectural design which includes a central cluster consisting of three specialized nodes to distribute tasks, responsibilities and maintain a loosely coupled architecture throughout. A pictorial representation of the Cluster-Node-Actor hierarchy in the system is shown through the diagram below:



As shown, the central Akka Cluster consists of three specialized nodes handling specific unique tasks.

Node 1: Core Services Node

This node consists of an ActorSystem called DatabaseExplorationSystem which acts a pivotal part of the architectural flow of dbVybe. This node is responsible for user authentication, session management, onboarding, core system coordination and database connections. It is configured to run on port 2551 in this application. Below are the Actors present in this node:

1. **SecurityActor** - Handles authentication, authorization, and security policies
2. **UserSessionManager** - Manages user sessions, login/logout, and session state
3. **DatabaseCommunicationManager** - Manages database connections (PostgreSQL, MySQL, MongoDB)

Apart from these, there is another actor called **DatabaseExplorationSupervisor** which acts as the root supervisor actor for the core services node.

Node 2: LLM Processing Node

This node contains an ActorSystem called LLMProcessingSystem that forms the core backend or the processing portion of dbVybe as it handles natural language processing and understanding, database query generation from natural language, LLM model management and coordination along with intelligent request routing. It is configured to run on port 2552 in this application. Below are the Actors present in the node:

1. **LLMOrchestrator** - Coordinates between different LLM components



2. **LLMACTOR** - Intelligent request router that determines if a message needs query generation or general chat
3. **QueryGenerationActor** - Generates SQL queries from natural language using AI agents
4. **QueryExecutorActor** - Executes generated queries and returns results

Apart from these actors, this node also contains a root supervisor for coordination called **LLMProcessingSupervisor**.

Node 3: Data Analysis Node

This node contains an ActorSystem called DataAnalysisSystem that runs various analytical services for dbVybe by performing database schema analysis, vectorization, pattern recognition and insights generation and graph-based relationship analysis. It is configured to run on port 2553 in this application. Below are the Actors present in the node:

1. **SchemaAnalysisActor** - Analyzes database schemas and creates vector embeddings
2. **VectorizationActor** - Handles semantic search by computing similarities
3. **GraphActor** - Manages graph-based relationship analysis

Apart from the above actors, this node also has a root supervisor actor for data analysis called **DataAnalysisSupervisor**.

AI Agents and Tools used by each Node in the Architecture

Core Services Node

- a. Handles database connections using JDBC driver for MySQL to connect to user database for user account management
- b. Uses MySQL database for persistent session management
- c. JWT token validation and user authentication for security

LLM Processing Node

- a. Uses **NLPAgent** for natural language processing and understanding for query generation and general chat
- b. Uses **Groq Inference Engine** via external APIs accessed asynchronously via **Spring WebClient** for LLM processing. Groq is configured to use **OpenAI's GPT-OSS 120b** model for reasoning
- c. **QueryExecutionAgent** is used as a core AI agent to handle safe execution of database queries on the connected database environment. This ensures no updates are made to the database via the chat interface and dbVybe is used strictly for database exploration, understanding and analytical purposes only



- d. **Langchain** (via *Langchain4j*) is used as a core AI framework in dbVybe to build and orchestrate various AI Agents according to the use cases. It helps in managing AI agents, handle memory management, prompt engineering, provides embedding model integration and message classification

Data Analysis Node

- a. **DatabaseSchemaAgent** is used by this node to analyze database schema of the connected database (currently supports *PostgreSQL*, *MySQL* and *MongoDB*, but is designed to be extendable to more databases in the future seamlessly) and creates vector embeddings
- b. **VectorAnalysisAgent** is used by the actors in this node to perform semantic searches to provide enhanced context to the LLM to answer user queries better
- c. **GraphAnalysisAgent** is also used to provide more context to the LLM as it stores complex database relations in a graph database to let the LLM know more about the connected database
- d. **Qdrant Vector Database** is used to store the vector embeddings of the database schema and **Neo4j Graph Database** is used to store the database relations
- e. **all-miniLM-L6-V2** embedding model is provided by **Langchain** to be used as the core embedding model

Tools Used for Persistence and Storage

As discussed earlier, dbVybe currently supports PostgreSQL, MySQL and MongoDB to be connected to the system for exploration but its modular, extensible and loosely-coupled architecture allows seamless development and integration of more databases in the future.

The dbVybe ecosystem itself uses three main database systems for persistence and processing data that flows through its pipeline:

- **MySQL:** dbVybe stores its user data such as credentials, sessions and database connection details in a MySQL instance running locally via Docker for this setup
- **Qdrant:** Embeddings of the database schema generated using Langchain's all-miniLM model are stored in a cloud instance of the Qdrant vector database for future processing and semantic searches
- **Neo4j:** Database analysis processes infer and store complex database relationships in a cloud instance of Neo4j graph database

Communication Protocols

As the dbVybe architecture involves many components along with a core Akka Typed framework for concurrency, it uses many different communication protocols for various use cases.

- **HTTP/REST:** All frontend to backend communications are handled through REST APIs over HTTP
- **Akka Messaging:** The core communication protocol used in the Akka Actor Model is the Akka Messaging protocol which facilitates inter-actor communication, actor-supervisor communication and cross-node communication
- **gRPC:** Qdrant database uses gRPC to connect and communicate with its processes and instance
- **JDBC:** JDBC is used to connect the app components with the SQL-based databases used in the system. This holds true for both the user connected SQL databases as well as the MySQL instance that dbVybe uses for its operations
- **MongoDB Wire Protocol:** In order to allow users to connect MongoDB to dbVybe, processes in the MongoDB analysis workflows use parts of the Wire protocol
- **Neo4j Bolt Protocol:** As dbVybe uses Neo4j for its operations, the workflows use methods and utilities provided by the Bolt protocol for communication with Neo4j instance
- **Spring WebClient:** dbVybe communicates with the Groq Inference Engine to access the LLM model via APIs but in order to asynchronously handle the communication in a non-blocking way, it uses Spring WebClient provided by the *WebFlux* framework

Actor Communications in Akka Cluster

dbVybe's Akka Cluster has three nodes with various Actors defined for various use cases as shown previously. These actors talk to each other and are connected to one another for various purposes in the application workflow.

- **Security Actor** - connected to *UserSessionManager* for session validation
- **UserSessionManager** - connected to *SecurityActor* for authentication
- **DatabaseCommunicationManager** - connected to *SchemaAnalysisActor* for schema analysis workflows
- **LLMOrchestrator** - connected to *LLMACTOR* for message processing
- **LLMACTOR** - connected to *QueryGenerationActor* to generate database queries and connected to *LLMOrchestrator* for coordination of conversations
- **QueryGenerationActor** - connected to *LLMACTOR* to receive processing requests
- **QueryExecutorActor** - connects to AI agents to execute queries

- **SchemaAnalysisActor** - connected to *DatabaseCommunicationManager* for database access
- **VectorizationActor & GraphActor** - connects to AI agents

Future Enhancements

- Extend support to more database systems
- Fine-tune LLM to maintain longer and clearer context
- Add more functionalities such as report generation, API pagination and response sharing
- Test the operation on various LLMs and prepare benchmarks to select the best LLM for the job
- Add a caching layer for quick retrieval of the data
- Add chat persistence to enhance context management across sessions

Conclusion

dbVybe has been designed for everyone to explore a range of database systems with the use of natural language and allow users without strong database and query language knowledge to be able to analyze, understand and study their database systems easily.

The core architecture relies on being distributed in nature and has been designed for scale with the use of Akka Framework that works on Actor Model for concurrency. The application is loosely-coupled to allow seamless debugging and swift feature enhancements in the future with the help of Akka Cluster architecture that uses multiple nodes for various operations. The heart of the application lies in the deep learning reasoning models that we know as Large-Language Models that allows strong reasoning. Context building is a crucial step for LLMs, which are done using various techniques including using Qdrant and Neo4j to enhance context for LLM to provide accurate reasoning results.

At last, the whole complex solution built for dbVybe is abstracted and provided as a simple, yet intuitive user-interface designed using React, Material-UI, Emotion and Lucide React and developed using Typescript and Vite for build tooling.

With the use of a strong infrastructure, dbVybe proves to be a great tool for database exploration and promises great future enhancements for real-world usage.

GitHub Repo: <https://github.com/Gizmosoft/dbvybe>